Progetto Scalable and Cloud Programming

Anno Accademico 2025-26

1. Descrizione

L'obiettivo del progetto è quello di realizzare una implementazione in Scala + Spark di un'analisi di co-occorrenza di terremoti in località diverse su un dataset specifico. Nello specifico, l'analisi consiste nel ricercare la coppia di locazioni con il massimo numero di co-occorrenze, ed elencare le date in cui co-occorrono gli eventi sismici in ordine crescente.
Al presente documento vengono allegati due dataset:

- versione ridotta
- versione completa

Il progetto su Google Cloud **deve usare la versione completa del dataset**. La versione ridotta vi sarà utile solo per fare dei test in locale sul vostro computer, o sulla configurazione **solo master** su Google Cloud.

1.1. Formato del Dataset

Poichè non siamo interessati alla specifica latitudine e longitudine del luogo in cui avviene un terremoto, ma piuttosto all'area in cui esso avviene, vogliamo approssimare la latitudine e la longitudine *alla prima cifra decimale*, con **arrotondamento all'intero più vicino**. Per esempio, le seguenti coppie di latitudine e longitudine ricadono tutte nella coppia `(112.2, 81.6)`.

```
112.234, 81.593
112.237, 81.596
112.199, 81.622
```

Allo stesso modo, non siamo interessati alla co-occorrenza nello stesso istante temporale, ma piuttosto alla co-occorrena in una finestra temporale. In particolare, **vogliamo che la finestra temporale sia di un giorno**. I seguenti istanti, ad esempio, ricadono tutti nella data `2025-01-01`

```
2025-01-01 00:00:01
2025-01-01 12:00:00
2025-01-01 23:59:59
```

1.2. Formato del risultato dell'analisi

Per quanto riguarda il formato dell'analisi, siamo interessati ad ottenere la lista di date in **ordine crescente** in cui co-occorrono due eventi in località differenti. Dato il seguente esempio,

latitude	longitude	date
37.502	15.312	2024-03-12 02:10

latitude	longitude	date
38.112	13.372	2024-03-12 05:55
37.521	15.324	2024-03-12 04:32
37.547	15.323	2024-04-01 14:22
38.147	13.324	2024-04-01 21:55
37.535	15.341	2024-04-03 12:32
38.142	13.387	2024-04-03 18:33
43.769	11.255	2024-04-03 21:10
37.522	15.308	2024-04-23 02:10
38.123	13.379	2024-04-27 05:55

la coppia `((37.5, 15.3), (38.1, 13.4))` è la coppia che co-occorre più spesso (le coordinate vengono arrotondate alla prima cifra decimale), e le date in cui co-occorrono i terremoti sono:

- 2024-03-12
- 2024-04-01
- 2024-04-03

Il formato di output richiesto comprende la coppia che co-occorre più spesso, e la lista di date in cui co-occorrono gli eventi. L'output sull'esempio precedente sarà il seguente:

```
((37.5, 15.3), (38.1, 13.4))
2024-03-12
2024-04-01
2024-04-03
```

Attenzione

È necessario rimuovere eventuali duplicati che si possono presentare per via dell'arrotondamento. Nell'esempio precedente, il primo ed il terzo evento vengono trattati come unico evento avvenuto il 2024-03-12 nella "cella" `(37.5, 15.3)`! Senza questo accorgimento, si rischia che le due località che co-occorrono più spesso hanno le stesse coordinate!

1.3. Implementazione

L'elaborazione dei dati deve essere fatta in modo distribuito, deve essere programmata usando l'approccio map-reduce, ed eseguita su piattaforma Apache Spark su DataProc. Si consiglia di provare diversi approcci confrontandone le prestazioni. Per valutare prestazioni e scalabilità bisogna fare diverse prove con cluster di dimensioni differenti.

2. Consegna

Il progetto è individuale e deve essere consegnato entro il 15 Ottobre 2026 (data di scadenza degli education credit di Google Cloud Platform). La consegna si effettua tramite il sito "Virtuale" del corso caricando un report in PDF di massimo 5 pagine in cui si descrive il lavoro svolto, l'approccio utilizzato, ed una analisi di scalabilità e prestazioni. Nel report va inserito un link ad un repository pubblico su GitHub dove deve essere presente il codice sorgente del progetto con un README contenente le istruzioni per

eseguire il programma su DataProc. Una volta fatta la consegna, bisogna darne comunicazione via e-mail a: nicolo.pizzo2@unibo.it e gianluigi.zavattaro@unibo.it.

3. Valutazione

La valutazione del progetto è in un range $[-2, 2]$ e verrà **sommata** al risultato parziale dello scritto. La valutazione terrà conto di:

- Approccio utilizzato
- Completezza del report
- Scalabilità
- Prestazioni

4. Comandi utili

Per leggere un file csv come `RDD` su Spark, bisogna invocare la funzione `read`, con delle opzioni specifiche per leggere l'header del file CSV.

```
val spark = SparkSession.builder
      .appName("Earthquake Application")
      .getOrCreate()

val filename = "..."
val data = spark.read.option("header", value = true).csv(filename).rdd
```

Potete interagire con DataProc sia tramite l'interfaccia web di Google Cloud Platform, sia da terminale. Da terminale è necessario installare il tool `gcloud` e autenticarsi con il proprio account Google (con gcloud auth login). Vengono ora elencati alcuni comandi da terminale che possono essere utili per interagire con DataProc.

Per creare un nuovo cluster DataProc:

```
gcloud dataproc clusters create <nome> \
  --region=<regione> \
  --num-workers <n> \
  --master-boot-disk-size 240 \
  --worker-boot-disk-size 240 \
  --master-machine-type=n2-standard-4 \
  --worker-machine-type=n2-standard-4
```

Attenzione

In generale si consiglia di impostare un disk size (in questo esempio a 240GB) per evitare di superare il limite imposto agli education credit. Inoltre, dovrete **obbligatoriamente** configurare in modo esplicito il tipo di macchina, usando il tipo `n2-standard-4`. Questo si può fare **solo da CLI** ed è, ad oggi, l'unico modo in cui potrete creare un cluster con 4 nodi worker. Per il numero di workers si consiglia di fare diverse prove con **cluster di 2, 3 e 4 workers**. Si consiglia di testare inoltre diverse configurazioni sul numero di partizioni adatto a ciascuna tipologia di cluster, utilizzando il metodo `repartition`.

Per immettere un job:

```
gcloud dataproc jobs submit spark \
--cluster=<nome> \
--region=<regione> \
--jar=gs://<bucket>/<nome-jar>.jar
```

Quando si vuole lanciare il proprio programma su DataProc, potete compilare un JAR e caricarlo su un vostro bucket in Google Cloud Storage precedentemente creato. DataProc è in grado di interagire con i bucket di Google Cloud Storage, quindi basta specificare l'URI appropriato. Questo vale anche per la lettura di dataset (come un file csv) caricato su un bucket e letto via codice Spark tramite l'URI del bucket e infine eseguito su DataProc.

Per rimuovere un cluster:

```
gcloud dataproc clusters delete <nome> --region <regione>
```